

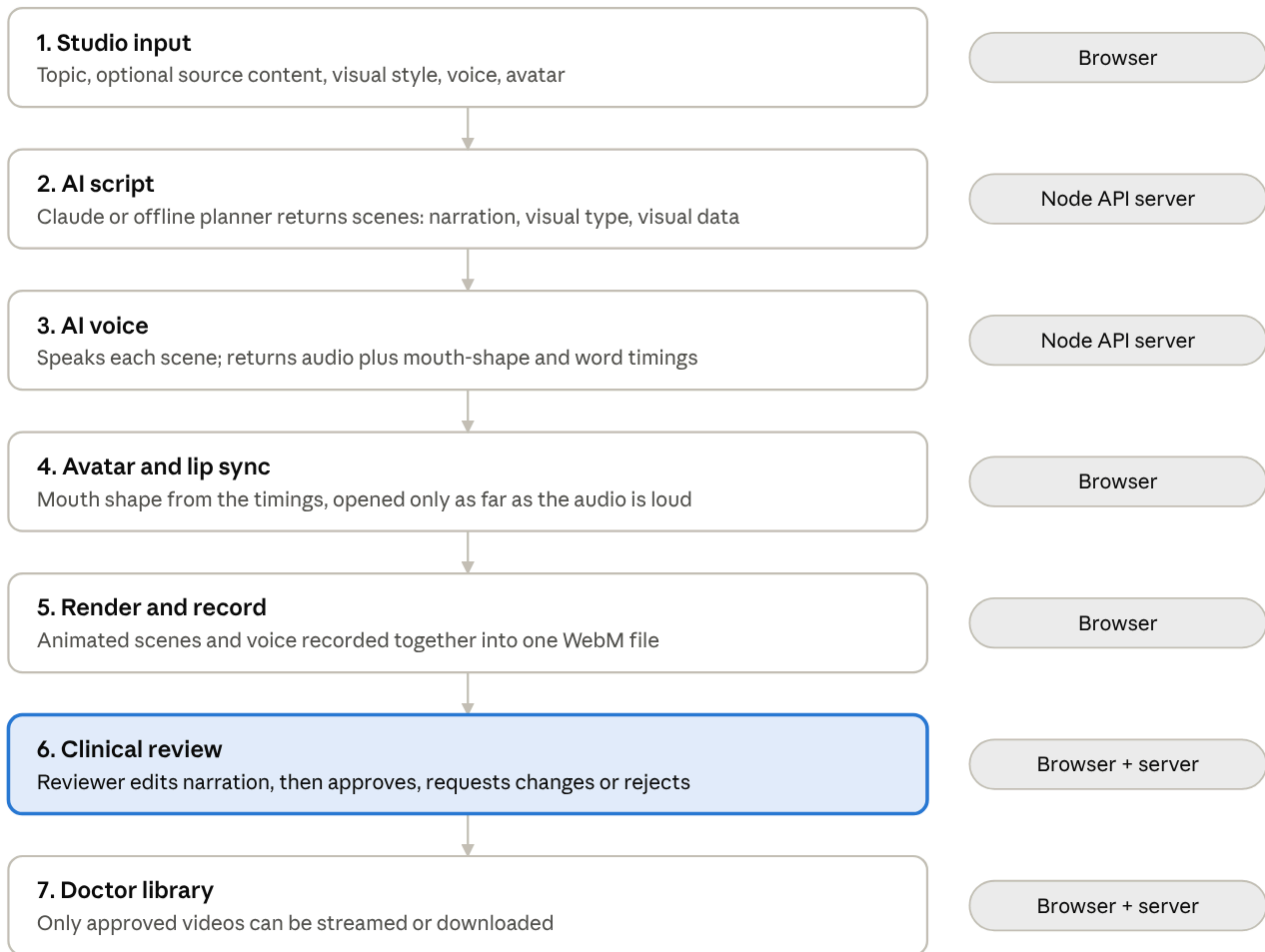
How the AI Medical Video Generator Works

Sep 30, 2026 · @Someone

At a glance

The server writes the script and the voice; the browser animates the avatar, syncs its lips and records the file; nothing reaches doctors without approval.

Every video is AI-generated end to end, then gated by clinical review



generation pipeline · 7 steps

The tag on the right of each step shows where it runs.

Step 1: AI script

The server turns a topic into a storyboard of 3 to 8 scenes, each saying what the presenter speaks and what the screen shows.

- **With `ANTHROPIC_API_KEY`** : Claude writes the storyboard against a strict JSON schema. It can only return known fields and the six supported visual types, so the renderer never receives something it cannot draw. It also lists claims the reviewer should verify.
- **Without a key**: an offline planner splits the source content into key points, or uses a standard outline (introduction, core concept, how it develops, key points). It flags every storyboard for full clinical review.

Each scene carries:

Field	Example
<code>title</code>	Core concept
<code>narration</code>	The exact words the avatar will say
<code>visualType</code>	<code>avatar</code> , <code>whiteboard</code> , <code>diagram</code> , <code>comparison</code> , <code>medical-animation</code> or <code>anatomy</code>
<code>seconds</code>	Planned length; the real length comes from the voice audio
<code>visualData</code>	Heading, bullet points, steps, comparison columns or organ to animate

The content team can edit any scene in the Studio before generating the video.

Step 2: AI voice

Each scene's narration becomes real audio bytes on the server, plus a timeline of mouth shapes and words. The browser's built-in speech was dropped because its audio cannot be recorded into a file.

Engine	When it is used	Timing accuracy
ElevenLabs	<code>ELEVENLABS_API_KEY</code> is set	Exact: the engine reports when each character is spoken
macOS neural voices (<code>say</code>)	Default; offline, server on a Mac	Estimated: sounds spread across the measured audio length, weighted by how long each sound takes

The response from `POST /api/studio/voice` contains:

- **Audio:** the spoken narration (WAV from macOS, MP3 from ElevenLabs).
- **Viseme timeline:** a list of mouth shapes with start time and length, e.g. MBP at 0 ms for 73 ms, then DD, then U.
- **Word timeline:** when each word starts and ends, used for subtitles.

The audio and both timelines come from the same cleaned-up text, so they cannot drift apart.

Step 3: Avatar and lip sync

The mouth takes its shape from the viseme timeline and its openness from the audio's loudness. As a result, it forms the right sound and closes during pauses.

1. **Spelling to mouth shapes.** The server maps the narration's letters to 13 standard mouth shapes (visemes). It checks longest letter groups first, so "tion" becomes one SS shape rather than four.
2. **Shapes to time.** Each shape gets a start time and length from the voice engine's timings (see Step 2).
3. **Loudness envelope.** The browser measures the decoded audio's loudness 120 times a second. The mouth only opens as far as the voice is actually loud, so silence always means a closed mouth.
4. **Smooth transitions.** Shapes blend over 55 ms, the way a real mouth moves toward the next sound before finishing the current one.
5. **Drawing.** Each shape sets lip width, opening, rounding, visible teeth and tongue, and jaw angle.

Mouth shape	Sounds	Look
MBP	m, b, p	Lips pressed together
FV	f, v	Lower lip under upper teeth
TH	th	Tongue between teeth
SS	s, z, sh, ch	Narrow, teeth showing
DD	t, d, n, l	Tongue behind upper teeth
KG	k, g, ng, h	Slightly open
RR	r	Rounded and tense
AA	father, cat	Wide open
E	bed, say	Mid-open, spread

Mouth shape	Sounds	Look
I	sit, machine	Narrow, spread
O	go, thought	Rounded
U	boot, would	Tightly rounded
REST	silence	Closed, neutral

Beyond the lips. The avatar also blinks, moves its eyes, breathes and sways. Its head nods and its brows lift more when the voice is louder. The four avatar looks (Doctor, Consultant, Surgeon, Presenter) share the same face rig.

Step 4: Rendering and recording

The browser draws each frame on a canvas and records the picture and the voice into one WebM file (VP9 video plus Opus audio).

1. **Voice every scene first.** All narration is synthesised and decoded before recording starts. Each scene lasts exactly as long as its audio, plus a 320 ms pause between scenes.
2. **One stream for picture and sound.** The canvas is captured at 30 frames per second. The narration plays through Web Audio into a recordable audio track, and both tracks go into a single `MediaRecorder`.
3. **One clock.** Every frame reads the audio clock, not the wall clock, to decide which scene, mouth shape and subtitle word to draw. The picture therefore stays in sync with the voice for the whole video.
4. **Keeps rendering in the background.** A worker-based timer drives the frames, so recording continues at full rate if the user switches tabs.
5. **Finish.** The last frame is held briefly, the recorder flushes, and the file is ready to download or submit.

Scene type	What animates
Avatar	Presenter on the left, key points on the right
Whiteboard	Points appear one by one as they are spoken
Step diagram	Steps connected by arrows, revealed in order
Comparison	Two columns side by side
Medical animation	A disease progression shown stage by stage

Scene type	What animates
Anatomy	A moving organ: beating heart, breathing lungs, brain, kidney, liver, stomach, blood flow in a vessel, or a dividing cell

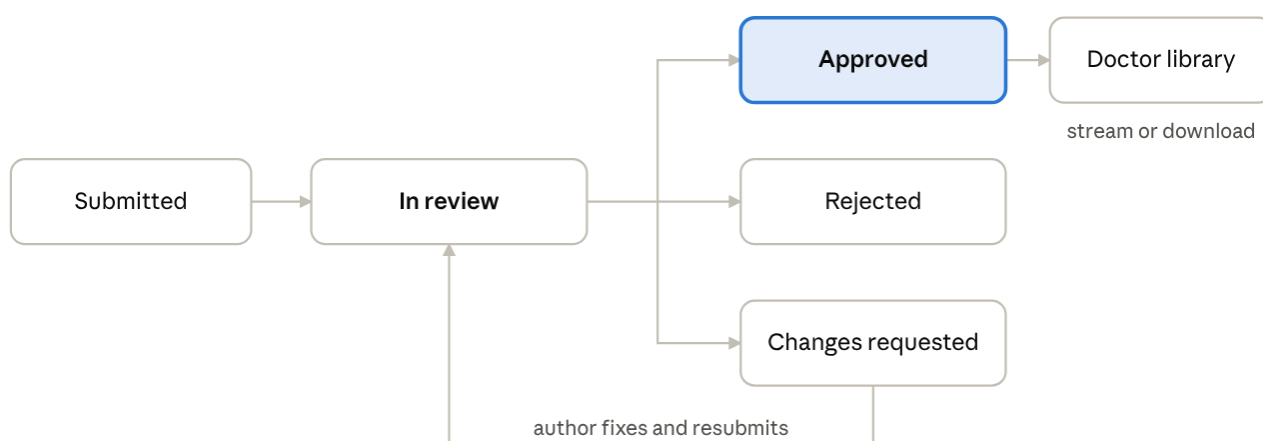
In anatomy scenes a small lip-syncing presenter stays in the corner. Word-by-word subtitles and a progress bar are drawn on every scene.

Output is 1280×720 at 3.5 Mbps or 1920×1080 at 6 Mbps, with 128 kbps audio.

Step 5: Review and distribution

The server enforces the approval gate: a video file is served to doctors only after a named reviewer approves it.

Only an approved video reaches the doctor library



review statuses · 5 states

A request for any unapproved video file returns HTTP 403. Reviewers get a preview-only exception. Every decision is stored with the reviewer's name, notes and time, and appended to the video's history.

Known gap: a reviewer can edit narration and approve in the same step. The narration text is updated, but the video file still has the old audio. The fix is to block approval until the video is regenerated after an edit.

Endpoint	Purpose
GET /api/studio/capabilities	Which script and voice engines are active
POST /api/studio/script	Write a storyboard

Endpoint	Purpose
POST /api/studio/voice	Voice one scene: audio, viseme and word timelines
POST /api/library	Submit a video for review
PUT /api/library/:id/media	Upload the recorded file
POST /api/library/:id/review	Approve, request changes or reject
POST /api/library/:id/resubmit	Send back into review
GET /api/library?audience=doctor	Approved videos only
GET /api/library/:id/media?download=1	Download the file (approved only)

Code map

Server logic lives in `server/src` ; browser logic lives in `src/app` .

File	Role
<code>server/src/services/script-writer.js</code>	Claude storyboard writer and JSON schema
<code>server/src/services/local-planner.js</code>	Offline storyboard planner
<code>server/src/services/tts/index.js</code>	Picks the voice engine; returns audio plus timelines
<code>server/src/services/tts/elevenlabs.js</code>	ElevenLabs voice with character timings
<code>server/src/services/tts/macOS.js</code>	Offline macOS voice
<code>server/src/services/visemes.js</code>	Letters to mouth shapes, placed on the audio timeline
<code>server/src/services/store.js</code>	Review statuses, history, video files on disk
<code>server/src/routes/studio.js</code>	Script and voice endpoints
<code>server/src/routes/library.js</code>	Review, upload, approval-gated streaming and download

File	Role
<code>src/app/core/services/video-producer.service.ts</code>	Voices scenes, syncs picture to audio, records the WebM
<code>src/app/core/services/lip-sync.service.ts</code>	Loudness envelope and mouth-shape blending
<code>src/app/core/services/avatar-renderer.service.ts</code>	Draws the four avatars: face, mouth, blinking, breathing
<code>src/app/core/services/scene-renderer.service.ts</code>	Draws the six scene types, subtitles and progress bar
<code>src/app/core/services/frame-clock.service.ts</code>	Worker-based frame timer for background tabs
<code>src/app/features/studio</code>	Studio screen
<code>src/app/features/review</code>	Review queue screen
<code>src/app/features/library</code>	Doctor library screen